

From the command line, I tested the template string feature. To exit from the command line, type `.exit` (dot exit).

Here's a sample snippet of how node.js will behave for the same input.

```
janardan@ubuntu:~$ node --version
v0.12.6
janardan@ubuntu:~$ node
> greet = "Hello, node.js";
'Hello, node.js'
> console.log('Message: ${greet}');
...

```

In the last statement, as node.js does not understand *backtick*, the node continues to expect input from the user by giving ellipses.

This month, I cover topics that are really additions to existing ES5, rather than entirely new features.

The literals are supported as part of the theme, where JS should go low-level (as in close to the machine and bits). This is useful for programmers doing bit manipulations, such as file permissions or network packet processing.

The array, string and math library is to eliminate the need for programmers to use other libraries (such as *Underscore.js*), or to code their own helper functions.

Binary and octal literals

There are two new number literals that have been added to JavaScript. These are the binary and octal literals. This is actually a re-introduction from previous releases.

The *0b* (zero-b) prefix is for the binary numbers and *0o* (zero-oh) prefix is for octal numbers.

```
> 0b111 // Output: 7 (decimal)
```

The output is in decimal.

```
> 0b11+0o11 // Output: 12 (3+9)
```

The following is the sum of the binary literal and the octal literal. The result is decimal.

```
> 0b11+0o11+0x11+011 // Output: 38
```

The following is the sum illustrating binary, octal, hex and octal. Note that JS interpreters still support *0* (zero) prefix notation for octal.

```
> "use strict"; 0b11+0o11+0x11+011
SyntaxError: Octal literals are not allowed in strict mode.
```

When you run the same in *use strict* mode, you get an error as only the *0* (zero) prefix octal is deprecated.

Enhanced object literals

Enhancements to object literals involve a cluster of features rather than a single one. The first is shorthand for object properties at the time of initialisation. Shown below is an example that best explains it. The second enhancement is the shorthand for object methods, and the third is computed property names.

Properties shorthand

In ES5, when we constructed object properties with scalar variables, we repeated the variable names like what's shown below in Snippet 1:

```
let firstname = "Janardan",
    lastname = "Revuru",
    organization = "HP EG India R&D";

let employee = {
  firstname: firstname,
  lastname: lastname,
  organization: organization,
};
```

Snippet 1: Longform notation

In ES6, we can avoid the repetition of object properties and variables names, if they are the same, as shown below in Snippet 2:

```
let employee = {
  firstname,
  lastname,
  organization,
};
```

Snippet 2: Shorthand notation

You can even mix shorthand notation with direct initialisation, like what's shown below in Snippet 3:

```
let employee = {
  firstname,
  lastname,
  org: organization,
  location: "Mahadevapura"
};
```

Snippet 3: Mixed notation

Method shorthand

ES6 also provides shorthand notation for the method definitions. You can eliminate the keyword 'function' when defining method properties.

Extending Snippet 2, we can write the code as shown